

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

И. Г. Криволапчук

МЕХАТРОНИКА: ИНТЕРФЕЙСЫ

Лабораторный практикум



Санкт-Петербург
2025

УДК 621
ББК 31.261
К82

Рецензенты:
кандидат технических наук, доцент *С. В. Быстров*;
старший преподаватель *Н. В. Решетникова*

Утверждено
редакционно-издательским советом университета
в качестве лабораторного практикума

Протокол № 2 от 26 марта 2025 г.

Криволапчук, И. Г.
К82 Мехатроника: интерфейсы: лаб. практикум / И. Г. Криволапчук. – СПб.: ГУАП, 2025. – 53 с.

Практикум содержит теоретические сведения и рекомендации по выполнению лабораторных работ с использованием тренажера QNET Mechatronic Interfacing.

Издание предназначено для практико-ориентированной подготовки студентов по направлениям подготовки 27.04.04 – «Управление в технических системах», 15.03.04 – «Автоматизация технологических процессов и производств».

УДК 621
ББК 31.261

Учебное издание

Криволапчук Игорь Григорьевич
МЕХАТРОНИКА: ИНТЕРФЕЙСЫ
Лабораторный практикум

Публикуется в авторской редакции
Компьютерная верстка *В. Н. Костиной*

Подписано к печати 23.09.2025. Формат 60 × 84 1/16.
Усл. печ. л. 3,1. Уч.-изд. л. 3,2.
Тираж 50 экз. Заказ № 231.

Редакционно-издательский центр ГУАП
190000, г. Санкт-Петербург, ул. Большая Морская, д. 67, лит. А

© Санкт-Петербургский государственный
университет аэрокосмического
приборостроения, 2025

ВВЕДЕНИЕ

Мехатроника – область науки и техники, основанная на синергетическом объединении узлов точной механики с электронными, электротехническими и компьютерными компонентами, обеспечивающими проектирование и производство качественно новых механизмов, машин и систем с интеллектуальным управлением их функциональными движениями.

Одними из важных элементов любой мехатронной системы являются интерфейсы, во многом определяющие параметры и возможности всей системы. Предлагаемый цикл лабораторных работ позволяет познакомиться с некоторыми распространенными типами интерфейсов как для внутри-, так и для межблоковой связи компонентов мехатронных систем – датчиков, накопителей информации, элементов человеко-машинного взаимодействия.

В качестве технической базы лабораторных работ используется персональный компьютер, взаимодействующий с многофункциональной лабораторной станцией NI ELVIS[1] в сочетании со сменной панелью тренажера QNET Mechatronic Interfacing [2, 3]. Управление работой тренажера, сбором данных и взаимодействием с пользователем реализуется библиотекой модулей, построенных на основе системы графического программирования LabVIEW [4].

Тренажер Quanser QNET Mechatronic Interfacing, изображенный на рис. В.1, является универсальной мехатронной цифровой интерфейсной системой, разработанной для обучения и демонстрации основ встраиваемых систем. Компактная система на базе микроконтроллера dsPIC33e включает в себя ряд устройств I2C и SPI, таких



Рис. В.1. Общий вид тренажера QNET Mechatronic Interfacing

как датчики температуры и ускорения, микросхемы памяти, интерфейсные устройства и внешние порты связи для UART и CAN. Лабораторная станция NI ELVIS II⁽⁺⁾ предоставляет простой интерфейс через виртуальные приборы LabVIEW, с помощью которого внутренние регистры микроконтроллера можно изменять в реальном времени для управления поведением всей системы без необходимости повторной прошивки.

Перечень компонентов тренажера, обозначенных на рисунке:

1. Потенциометр (источник аналогового сигнала)
2. Микросхема flash-памяти с интерфейсом SPI
3. Микросхема RAM с интерфейсом SPI
4. Регулируемые резисторы «подтяжки» для интерфейса I²C
5. Микроконтроллер
6. Индикатор питания +5 В
7. Кнопка сброса системы
8. Инкрементные энкодеры
9. Слот для SD-накопителя
10. Контроллер трехцветного светодиода с интерфейсом SPI
11. Датчик цветового излучения
12. Контроллер сенсорного экрана
13. Цифровая клавиатура
14. Оптический канал последовательной передачи (UART)
15. Акселерометр с интерфейсом I²C
16. MEMS-гироскоп с интерфейсом I²C
17. Датчик приближения с интерфейсом I²C

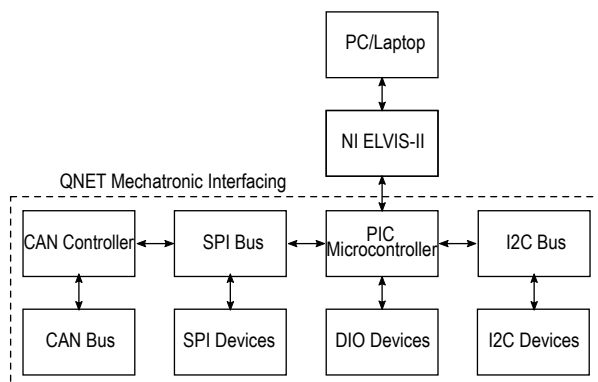


Рис. В.2. Взаимодействие между компонентами QNET Mechatronic Interfacing

18. ЖК-экран с сенсорным вводом
19. Разъем 0 шины CAN
20. Разъем 1 шины CAN
21. Датчик температуры с интерфейсом I²C.

Взаимодействие между различными компонентами лабораторной установки показано на рис. В.2.

Станция NI ELVIS II подключается к ПК или ноутбуку через USB-соединение. Плата QNET Mechatronic Interfaceing обеспечивает интерфейс связи с NI ELVIS II для непосредственного воздействия на регистры управления и работу микроконтроллера практически так же, как если бы он выполнял команды непосредственно из памяти программ. Это, в свою очередь, позволяет осуществлять прямое управление интерфейсом между микроконтроллером и остальными компонентами на плате.

1. ОБЩИЕ СВЕДЕНИЯ ОБ ИССЛЕДУЕМЫХ ИНТЕРФЕЙСАХ

1.1. Интерфейс ввода/вывода микроконтроллера

В QNET Mechatronic Interfacing используется микроконтроллер из семейства устройств Microchip™ dsPIC33e (PIC)[5].

Микроконтроллеры – это миниатюрные вычислительные устройства, в которых центральный процессор (ЦП), оперативное и долговременное запоминающие устройства и контроллеры ввода/вывода объединены в одну микросхему. Инструкции загружаются в программную память, и ЦП выполняет их последовательно. QNET Mechatronic Interfacing настроен так, чтобы LabVIEW™ предоставлял команды ЦП, как если бы они были извлечены из программной памяти.

Регистры

Поскольку ЦП выполняет инструкции в программной памяти, он сохраняет данные в специализированных ячейках памяти, называемых регистрами. Некоторые из этих регистров управляют внутренней функцией микроконтроллера, в то время как другие хранят информацию, входящую или исходящую из контроллера через периферийные устройства, такие как порты ввода/вывода или аналого-цифровые преобразователи.

Цифровой ввод/вывод

Микроконтроллеры управляют цифровым вводом и выводом через порты ввода/вывода. Это наборы регистров, которые постоянно считываются и обновляются для отражения состояния сигнала физических контактов на чипе. На микроконтроллерах PIC эти порты ввода/вывода идентифицируются буквой алфавита, начиная с порта А, затем порта В и тд. Каждый из этих портов имеет несколько регистров, которые можно использовать для контроля и изменения состояния сигнала соответствующего контакта. Для каждого регистра младший значимый бит будет обозначаться как bit 0, а старший значимый бит – bit 15.

Самым простым из этих регистров является регистр PORTx. Этот регистр отражает фактическое состояние контакта в любой момент времени. Чтение PORTA вернет текущее цифровое состояние (высокое или низкое) выводов, обозначенных RA0-RA15, то есть физических выводов, связанных с портом А. Например, если PORTA считывается и возвращает значение 0x10C8 или 0b0001 0000 1100 1000, это означает, что в настоящее время RA3, RA6, RA7 и RA12 имеют высо-

Таблица 1.1

Требуемое состояние	PORTx	LATx	TRISx
Цифровой выход HIGH	1	1	0
Цифровой выход LOW	0	0	0
Цифровой вход HIGH	1	x	1
Цифровой вход LOW	0	x	1

кий уровень, а все остальные линии имеют низкий уровень. Важно помнить, что регистры и линии данных индексируются нулем.

Далее идет регистр LATx или защелка. Этот регистр хранит выходное значение, назначенное выводам в порту. Чтение LATA вернет значения, которые будут выводиться на выводах RA0-RA15, если эти выводы настроены как выходы. Запись в регистры PORTx или LATx изменит зафиксированное значение в регистре LATx для этого порта.

Третий базовый регистр управления портом – это регистр TRISx. Этот регистр определяет, сконфигурированы ли контакты как входы или выходы. В регистре TRISA каждый бит управляет направленностью одного контакта порта A. Если бит RA0 (бит 0) в TRISA установлен в «0», то контакт сконфигурирован как выход, и микроконтроллер попытается установить состояние этого контакта в соответствие со значением, сохраненным в бите RA0 LATA. Однако, если бит RA0 в TRISA установлен в «1», то контакт сконфигурирован как вход и его состояние может свободно изменяться внешними источниками. Выход защелки переходит в состояние высокого выходного сопротивления (Z-состояние) и не оказывает влияние на состояние контакта.

Сочетания значений управляющих битов для различных желаемых состояний конкретного контакта приведены в табл. 1.1.

Шум сигнала и гистерезис

Цифровые входы на микроконтроллере имеют порог переключения, который представляет собой аналоговое напряжение, выше которого вход считывается как HIGH («1»), а ниже которого он читается как LOW («0»). Это может вызвать проблемы, если из-за шума или проседания напряжения цифровой входной сигнал не явно выше или ниже этого порога. В этом случае состояние цифрового входа может неожиданно измениться, вызывая ошибки данных. Многие микроконтроллеры имеют входы, которые смягчают эту проблему путем введения гистерезиса во входной цепи. Это означает, что пороговое напряжение, при котором вход переключается из состояния OFF (выключено) в ON (включено), значительно выше, чем напря-

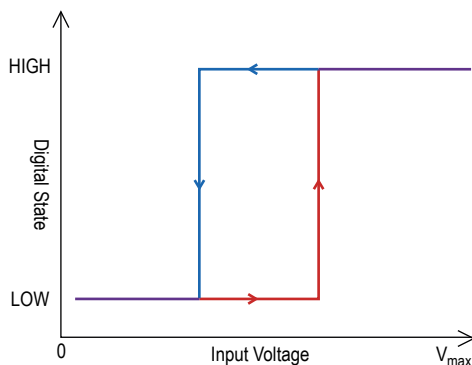


Рис. 1.1. Поведение цифрового входа с гистерезисом

жение, при котором он переключается из ON в OFF. Результатом является кривая отклика, похожая на показанную на рис. 1.1.

По мере увеличения напряжения на входе (красные линии) оно в конечном итоге достигает точки, в которой цифровое состояние переключается с низкого логического уровня (LOW) на высокий (HIGH).

Однако по мере уменьшения напряжения (синие линии) оно должно стать намного ниже, прежде чем цифровое состояние вернется в LOW. Это означает, что после изменения цифрового состояния шумовые колебания в напряжении сигнала с меньшей вероятностью приведут к неожиданному обратному изменению цифрового состояния. В контексте цифровых сигналов гистерезис количественно определяется как разница между напряжением, при котором сигнал перейдет из LOW в HIGH, и напряжением, при котором он перейдет из HIGH в LOW. Для примера, показанного на рис. 1.1, если вертикальная красная линия возникла при 2 В, а вертикальная синяя линия при 1.5 В, то гистерезис в системе составит 0.5 В.

Аналоговый вход

Аналоговый вход немного сложнее цифрового входа. Для того чтобы микропроцессор, который является цифровым устройством, обработал аналоговый вход, он должен преобразовать сигнал в цифровое значение, которое может быть сохранено в двоичной форме в памяти. Преобразование обычно выполняется аналого-цифровым преобразователем или АЦП. Существует много методов, с помощью которых можно выполнить аналого-цифровое преобразование, но почти все они приводят к двоичному значению, которое представляет собой часть известного опорного напряжения.

Микропроцессор dsPIC33е в составе QNET Mechatronic Interfacing имеет АЦП, настроенный на 12-битную работу с использованием опорного напряжения 3.3 В. Это означает, что при измерении напряжения на аналоговом входе АЦП генерирует 12-битное значение b , такое что

$$\frac{b \cdot 3.3V}{0xFFF} < V_{in} < \frac{(b+1) \cdot 3.3V}{0xFFF}.$$

Не все выводы порта можно настроить для аналогового входа. Обычно необходимо ознакомиться с техническим описанием микроконтроллера, чтобы узнать, какие контакты могут обрабатывать как цифровые, так и аналоговые входы. Управление тем, настроен ли вход как аналоговые входы или цифровой ввод/вывод, осуществляется путем установки регистра ANSELx для соответствующего порта.

Например, в случае семейства dsPIC33е, если бит RE6 в ANSELE установлен в «1», то это будет обычный цифровой ввод/вывод. Однако если бит RE6 в ANSELE установлен в «0», то он будет настроен как аналоговый вход, который можно оцифровать, назначив связанный аналоговый вход AN30 одному из внутренних АЦП.

В QNET Mechatronic Interfacing микропроцессор был предварительно настроен таким образом, что AN30, подключенный к потенциометру в верхнем левом углу платы тренажера, подключен к ADC1.

1.2. Последовательный периферийный интерфейс

Последовательный периферийный интерфейс, или SPI, – это последовательная коммуникационная архитектура, изначально разработанная Motorola для использования в собственных разработках на основе микропроцессорного семейства MC68k[6]. Это архитектура Master-Slave с одним ведущим устройством, которое управляет произвольным количеством ведомых устройств.

Типичные области применения включают сопряжение микроконтроллеров с периферийными микросхемами аналого-цифровых и цифро-аналоговых преобразователей, флэш-памяти и памяти EEPROM, а также различных микросхем связи.

Структура интерфейса

Все устройства в архитектуре SPI совместно используют одну линию синхронизации, которая контролируется исключительно ведущим устройством. Линия синхронизации сообщает всем устройствам, когда делать выборку входящих данных и когда менять исходящие данные. Эта линия обычно будет упоминаться в документации как SCLK, SCK

или CLK. Поскольку синхронизация всей системы управляется ведущим устройством, SPI считается синхронным интерфейсом.

SPI – это полнодуплексный интерфейс, что означает, что он имеет две отдельные линии данных, каждая из которых обрабатывает непрерывный поток данных в одном направлении. Первую линию данных можно однозначно назвать «Master Out, Slave In» или MOSI, и она используется для передачи данных от ведущего устройства ко всем ведомым устройствам. Эта линия может быть обозначена как SDO или Serial Data Out на главном устройстве или SDI или Serial Data In на подчиненных

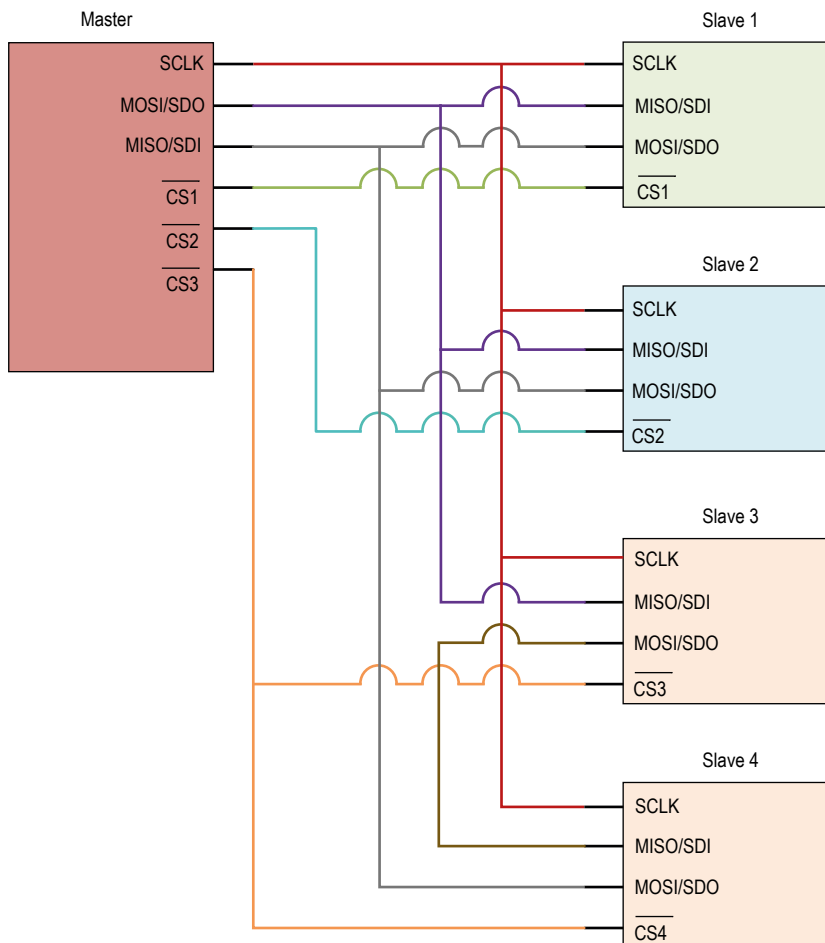


Рис. 1.2. Пример структуры шины SPI

устройствах. Вторая линия данных называется «Master In, Slave Out» или MISO и передает данные обратно от подчиненных устройств к главному. Во многих документах эта линия будет обозначена как SDI на главном устройстве или SDO на подчиненном устройстве. Будьте особенно внимательны при интерпретации функции линий SDO и SDI, поскольку они не используются одинаково на главных и подчиненных устройствах. Как показано на схеме примера на рис. 1.2, SDO на главном устройстве будет подключаться к SDI на подчиненном устройстве, и наоборот.

Поскольку линии данных подключены к нескольким подчиненным устройствам, необходимо предусмотреть возможность выбора того, какое устройство должно принимать и передавать данные по линиям данных в любой момент времени. С этой целью последняя линия в интерфейсе SPI уникальна для каждого подчиненного устройства (или группы последовательно соединенных подчиненных устройств) и называется выбором микросхемы или CS. Эта линия активна на низком уровне, что означает, что когда ведущее устройство хочет связаться с определенным ведомым устройством, оно должно установить линию выбора микросхемы для этого ведомого устройства на низкий уровень. Эту линию также иногда называют линией выбора ведомого устройства или линией SS.

Передача данных

Запись данных через SPI состоит из трех этапов. Сначала должна быть активирована линия CS для нужного ведомого устройства переводом её в состояние LOW.

Затем данные должны передаваться по одному биту за раз, в то время как линия CLK циклически повторяется дважды для каждого бита. Микропроцессор dsPIC33e из QNET Mechatronic Interfacing имеет встроенный модуль SPI, который автоматически обрабатывает этот процесс. После записи 8 байт данных в буфер данных SPI SPIxBUF модуль автоматически выводит данные на MOSI, одновременно тактируя линию синхронизации. Этот шаг можно повторять до тех пор, пока не будут отправлены все нужные данные.

Наконец, линия CS для ведомого устройства должна быть переведена в состояние HIGH, чтобы сообщить ведомому устройству о завершении передачи и необходимости действовать в соответствии с полученными данными.

Регистры устройств

Периферийные устройства имеют регистры данных, аналогичные тем, которые используются микроконтроллером. Данные, полу-

ченные устройством по линии MOSI, сохраняются в приемном буфере. После завершения передачи и возврата линии CS на высокий уровень эти данные считываются, и устройство действует в соответствии с этими данными. Формат данных, ожидаемых устройством, и поведение устройства после получения данных не стандартизированы, и эту информацию необходимо извлечь из спецификации устройства.

Конфигурация шины SPI

Подчиненные устройства могут быть подключены к ведущему устройству как независимо, так и в конфигурации последовательного соединения (так называемая «daisy chain»). На рис. 1.2 показан пример конфигурации, которая включает как независимые, так и последовательно соединенные подчиненные устройства.

На этом рисунке подчиненные устройства 1 и 2 независимы и могут напрямую взаимодействовать с ведущим устройством. В конфигурации независимого ведомого устройства линии MOSI и MISO подключены параллельно на всех устройствах, и каждое ведомое устройство имеет свою собственную линию выбора микросхемы.

Подчиненные устройства 3 и 4 соединены последовательно. Конфигурация последовательного соединения уменьшает количество контактов ввода-вывода на ведущем устройстве, которые должны быть выделены для выбора активного ведомого устройства, путем объединения линий выбора микросхемы для двух или более ведомых устройств вместе и последовательного соединения их линий MOSI и MISO. В конфигурации последовательной цепи выходные данные от первого подчиненного устройства в цепи подаются на вход следующего подчиненного устройства, поэтому может потребоваться добавление к передаваемым данным специальных заполнителей для «проталкивания» информации к целевому устройству.

Когда CS3 активируется, оба подчиненных устройства становятся активными. Когда затем данные отправляются по линии MOSI, они будут получены подчиненным устройством 3. В то же время подчиненное устройство 3 должно выводить данные, которые в данный момент находятся в его регистре данных SPI, на свой вывод MISO, чтобы их мог получить подчиненное устройство 4.

1.3. Интерфейс запоминающих устройств

Шина SPI всегда работает в полнодуплексном режиме, что означает, что каждый раз, когда ведущее устройство циклически переключает линию синхронизации, 1 бит информации отправляется на

MOSI, а один бит считывается с MISO. Как правило, ведомые устройства будут отправлять данные на линию MISO только после того, как им будет дана соответствующая команда. Когда полные 8 бит данных получены на линии MISO, модуль SPI переместит полученные данные в буферный регистр RX, где их можно будет прочитать. Поскольку протокол SPI определен лишь приблизительно и официально не стандартизирован, процесс чтения данных с подчиненного устройства будет сильно различаться в зависимости от рассматриваемого устройства. Как правило, необходимо обратиться к техническому описанию устройства, чтобы узнать, какие команды заставят его вернуть требуемые данные.

Для наглядной иллюстрации различий в работе со сходными по назначению устройствами используются имеющиеся в составе QNET Mechatronic Interfacing микросхемы памяти, которые можно использовать также для хранения данных. Первая микросхема – Microchip™ 23K640 SRAM, вторая – Spansion™ S25LF164K Flash. Обе микросхемы подключены к микроконтроллеру как независимые подчиненные устройства. Это означает, что у них обоих есть отдельные линии выбора микросхемы, и что выходные данные на их линиях SDO возвращаются непосредственно к главному устройству.

Оперативное запоминающее устройство

Оперативное запоминающее устройство или ОЗУ – это форма памяти для чтения/записи, обычно используемая для хранения краткосрочных данных во время выполнения других операций. Главное преимущество ОЗУ перед другими формами памяти заключается в том, что любое местоположение данных в ОЗУ может быть либо прочитано, либо записано примерно за одинаковое время. Для сравнения, большинство других форматов памяти либо не позволяют записывать данные (как в случае с постоянной памятью), либо требуют значительно больше времени для чтения и записи в определенные блоки памяти в зависимости от их физического расположения (как в случае с жесткими дисками).

В настоящее время все технологии ОЗУ являются энергозависимой памятью, что означает, что данные, хранящиеся в ОЗУ, теряются, как только устройство теряет питание. В промышленности распространены две формы ОЗУ: динамическое ОЗУ, или DRAM, и ОЗУ статического типа – SRAM.

Динамическое ОЗУ является самой простой формой оперативной памяти с точки зрения устройства битовой ячейки – в DRAM каждый бит информации хранится в виде заряда одного конденсатора в чипе ОЗУ.

Такая память дешева в производстве, однако, поскольку заряд конденсатора со временем стекает, данные будут потеряны, если заряд не обновляется периодически. С другой стороны, статическая оперативная память хранит каждый бит данных в бистабильной защелке (триггере). Это означает, что данные на чипе SRAM будут сохраняться до тех пор, пока схема находится под напряжением. Недостатком SRAM является то, что для хранения каждого бита требуется значительно больше элементов, поэтому она дороже и менее плотная, чем DRAM.

Флэш-память

Флэш-память – это специализированная форма электрически стираемой программируемой постоянной памяти (EEPROM). Флэш-память обычно отличается от других форм программируемой постоянной памяти тем, что ее можно стирать блоками или страницами, которые, как правило, намного меньше общего размера устройства, но намного больше одного байта. Это означает, что при записи данных нет необходимости стирать все устройство, но также нет необходимости стирать отдельное местоположение данных для каждого записываемого байта. Этот компромисс в размере страницы делает флэш-память быстрее для записи, чем другие устройства EEPROM, хотя и не так быстро, как ОЗУ.

Главное преимущество флэш-памяти перед ОЗУ заключается в том, что она энергонезависима и сохраняет данные даже при отсутствии питания. Это достигается путем хранения данных в состоянии специальных полевых транзисторов, известных как МОП-транзисторы с плавающим затвором. Эти транзисторы имеют особенности конструкции, поскольку должны сохранять свое состояние активации в течение очень долгого времени без подачи энергии.

К сожалению, хотя эта технология энергонезависима, у нее есть свои недостатки. Прежде всего, флэш-память можно записывать и считывать за постоянное время, как ОЗУ, а вот стирание флэш-памяти занимает гораздо больше времени. Это означает, что ячейки памяти не могут быть произвольно перезаписаны, как это возможно в ОЗУ.

Во-вторых, флэш-память имеет ограниченное количество раз, когда ее можно записывать и стирать, прежде чем ячейки памяти начнут изнашиваться. Это делает флэш-память непригодной для приложений, где данные будут часто перезаписываться.

1.4. Интерфейс связи интегральных компонентов

Шина Inter-Integrated Circuit, или I²C, – это асимметричный последовательный коммуникационный интерфейс, разработанный ком-

панией Philips Semiconductor в начале 1980-х как простая 8-битная шина внутренней связи для создания управляющей электроники, изначально рассчитанная на частоту 100 кГц. Архитектура Master-Slave позволяет строить системы с несколькими ведущими и несколькими ведомыми устройствами. Интерфейс I²C – это полностью документированный стандарт, первая версия которого появилась в 1992 году. В настоящее время актуальна 6-я версия стандарта, который регулируется NXP Semiconductors (ранее Philips)[7].

Шина I²C

I²C (I2C) – это полудуплексная последовательная коммуникационная шина. Это означает, что каждая линия может передавать информацию в двух направлениях, и направление потока данных меняется во время связи. Для связи I2C используются две линии шины, обе из которых контролируются в разное время либо ведущим, либо ведомым устройством.

Первая линия шины – это последовательная линия данных (SDA), которая передает данные между ведущим и ведомым устройствами. Эта линия будет контролироваться тем устройством, которое отправляет данные, однако направление потока данных всегда контролируется ведущим. В случае систем с несколькими ведущими устройствами это линия, которая используется для определения того, пытается ли другое устройство управлять шиной I2C как ведущим. Если это так, то модуль управления I2C прекратит попытки передачи и установит бит ошибки, чтобы указать на коллизию шины. QNET Mechatronic Interfacing имеет только одно ведущее устройство, поэтому акцент в первую очередь будет делаться на организацию связи в среде с одним ведущим устройством.

Вторая линия шины – это последовательная линия синхронизации (SCL), которая обычно управляется ведущим устройством, чтобы указать, когда байт данных записывается или считывается на линии SDA. В случае обычной передачи данных состояние SDA будет изменяться только, когда SCL низкий. Изменения в SDA, когда SCL высокий, интерпретируются как команды. Единственный раз, когда эта линия не управляется ведущим устройством, – это когда ведомое устройство удерживает линию синхронизации на низком уровне, чтобы указать, что оно не готово отправлять/принимать информацию.

Электрически обе линии SCL и SDA подключены к входам с открытым коллектором на всех устройствах, подключенных к шине. Это означает, что любое устройство на шине может использовать внутренний транзистор для подтягивания напряжения линии данных к земле.

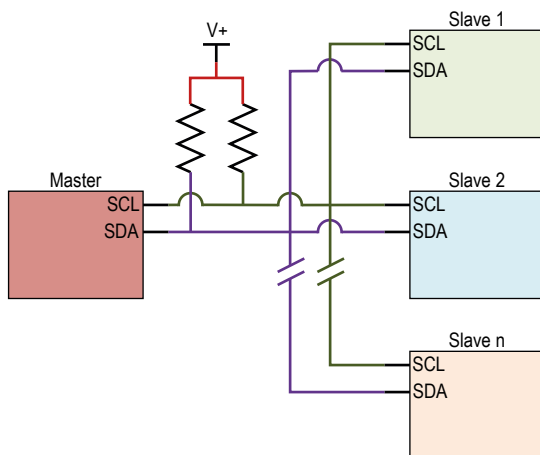


Рис. 1.3. Структура шины I2C

Однако ни одно устройство не может подать высокое напряжение на линию данных, вместо этого обе линии данных подключены к подтягивающим резисторам, которые удерживают линии на произвольном напряжении, когда ни одно устройство активно не понижает уровень линии. Электрическая структура шины I2C показана на рис. 1.3.

Передача данных

Прежде чем какие-либо данные могут быть переданы по шине I2C, ведущее устройство подтверждает, что обе линии SDA и SCL имеют постоянный высокий уровень. Это означает, что никакое другое устройство в данный момент не управляет шиной. Этот шаг важен, если есть вероятность, что более одного ведущего устройства могут пытаться управлять шиной. Пока линия SCL находится на высоком уровне, ведущее устройство понижает уровень линии SDA. Это называется условием запуска; команда, указывающая на начало передачи данных. Первый байт всегда передается ведущим устройством и содержит адресную информацию для передачи (см. ниже).

Для каждого бита данных, который должен быть передан, ведущее устройство понижает уровень линии SCL. Когда SCL низкий, передающее устройство устанавливает линию SDA в правильную полярность для этого бита данных. Затем мастер отпускает линию SCL в высоком состоянии, и приемное устройство считывает бит данных из SDA. Эта последовательность повторяется до тех пор, пока не будет передан полный байт (8 бит) данных.

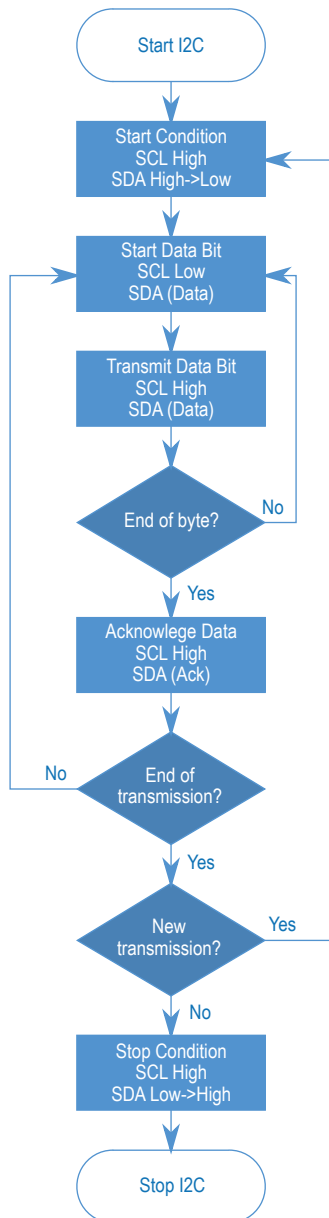


Рис. 1.4. Процесс связи I2C

После каждого байта передающее устройство позволяет SDA и SCL перейти в высокий уровень и ждет подтверждения от получателя. Получатель подтверждает получение полного байта, понижая линию SDA и оставляя SCL высоким. Если получателем передачи является мастер и дальнейшая передача не требуется, мастер вместо этого не подтвердит конец байта, оставив SDA высоким.

После того, как бит подтверждения отправлен/получен, мастер может немедленно начать передачу другого байта информации, понижая SCL и повторяя шаги для передачи данных. В качестве альтернативы, если мастер позволяет SCL оставаться на высоком уровне, он может указать на начало новой связи, переводя SDA с высокого на низкий уровень (называемый повторным условием запуска), или указать на конец передачи, отпуская SDA с низкого на высокий уровень (условие остановки).

Порядок событий при обмене по I2C показан на рис. 1.4.

Адресация I2C

Поскольку в шине I2C всего два провода, данные, отправленные по шине, принимаются всеми другими устройствами на шине таким же образом. Чтобы связаться с уникальным устройством, данные должны быть адресованы. Наиболее распространенная схема адресации включает 7-битное значение, которое является уникальным для конкретного устройства на шине. Адреса устройств жестко запрограммированы в устройствах, чтобы связь могла быть инициирована при первом подключении устройства к шине I2C. Эти адреса регулируются NXP Semiconductors, чтобы избежать конфликтов.

Некоторые устройства будут иметь один или несколько младших битов в своих адресах, которые можно установить, изменив полярность контактов на чипе, что позволяет нескольким устройствам одной модели сосуществовать на одной шине I2C.

Например, базовый 7-битный адрес датчика приближения ИК ADPS-9190 I2C в составе QNET Mechatronic Interfacing имеет значение 0x39 или 0b0111001. В табл. 1.2 показаны полные байты адреса, необходимые для связи с ИК-датчиком в зависимости от направления обмена.

Таблица 1.2

Адресация ИК-датчика приближения

Тип передачи	7-битный адрес	R/W Bit	Полный адрес
Чтение	0b0111001	1	0b01110011
Запись	0b0111001	0	0b01110010

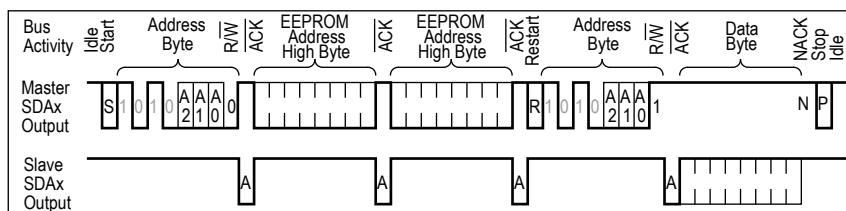


Рис. 1.5. Пример связи I2C

Обратите внимание, что в 7-битном адресе нет изменяемых битов, на шине I2C одновременно может присутствовать только одно устройство конкретной модели.

Первый байт данных, отправленных после условия запуска, должен состоять из 7-битного адреса, принадлежащего одному из устройств на шине, за которым следует один бит чтения/записи. Если за байтом адреса должны следовать дальнейшие данные, записанные в ведомое устройство, то бит чтения/записи должен быть низким. С другой стороны, если ведомое устройство должно отправить данные обратно ведущему устройству в следующем байте, бит чтения/записи должен быть высоким.

Рис. 1.5 иллюстрирует типичный сеанс связи между ведущим и ведомым устройством. В этом примере ведущее устройство сначала отправляет адрес для устройства (обратите внимание, что фактический адрес имеет переменные биты A0-A2, которые будут установлены на аппаратном уровне), при этом бит R/W низкий, что указывает на запись. Подчиненное устройство с соответствующим адресом устройства подтверждает получение. Следующие два байта – это данные, которые подчиненное устройство загружает в регистр устройства для последующего использования. Затем ведущее устройство инициирует повторное условие запуска и снова отправляет адрес, но на этот раз с битом чтения, указывающим, что подчиненное устройство должно отправить данные. Подчиненное устройство считывает адрес запрошенных данных из регистра устройства и отправляет запрошенный байт. Ведущее устройство, которое теперь является получателем данных, отвечает отрицательным подтверждением на данные, указывая, что больше данные отправляться не будут. Затем ведущее устройство инициирует условие остановки, чтобы завершить передачу.

1.5. Местная сеть контроллеров

Местная сеть контроллеров – Controller Area Network (CAN), – это стандарт связи, разработанный компанией Robert Bosch GmbH в се-

редине 1980-х, первая публичная спецификация которого опубликована в 1991 году [8]. В настоящее время CAN является стандартом de facto для автомобильной автоматики[9,10], а также имеет широкое распространение в промышленной автоматизации, технологиях домашней автоматизации («умный дом») и многих других областях.

QNET Mechatronic Interfacing имеет два внешних порта CAN, каждый из которых управляется отдельным чипом контроллера CAN. Эти порты соединены вместе, так что тренажер действует и как передатчик, и как получатель для всех коммуникаций по шине CAN. Однако эти порты также могут использоваться для связи с другим модулем QNET или доступными совместимыми периферийными устройствами.

Шина CAN

Сеть контроллеров, или шина CAN, физически состоит из двух проводников, образующих дифференциальную пару. Это означает, что данные передаются путем подачи на линии данных разности напряжений, а не сравнения напряжения любой из линий с общим опорным напряжением. Линии данных состоят из высокой (HI) и низкой линий (LO) данных, которые связаны между собой через резисторы. Когда шина CAN активна, эти линии находятся либо в доминантном, либо в рецессивном состоянии (см. табл. 1.3).

Стандарт CAN разработан таким образом, что если несколько устройств пытаются установить состояние линий данных, то доминирующее состояние (логический 0) победит. Это становится важным в процессе арбитража (см. ниже). Фактические напряжения линий данных и сопротивление оконечной нагрузки будут различаться в зависимости от того, какой стандарт физического уровня реализуется. Реализация шины CAN на QNET Mechatronic Interfacing представляет собой дифференциальную нагрузку 5 В с резисторами 120 Ом. Когда внешние разъемы CAN соединены вместе, полная сеть шины CAN показана на рис. 1.6.

Таблица 1.3

Состояния линий данных для различных значений битов

Состояние бита	Состояние линии данных HI	Состояние линии данных LO	Логическое значение бита
Доминантный	Высокий	Низкий	0
Рецессивный	Низкий	Высокий	1
Неактивный	Низкий	Низкий	Null

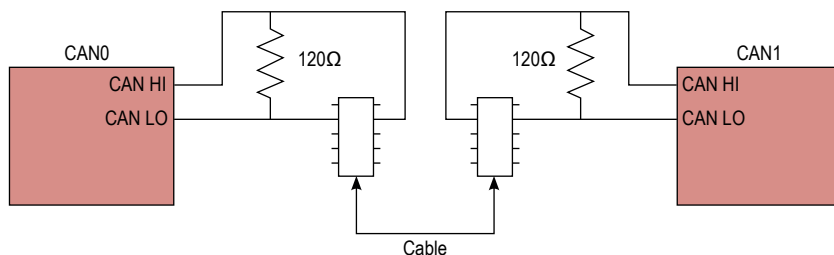


Рис. 1.6. Сеть CAN QNET Mechatronic Interfacing

CAN – это стандарт с несколькими ведущими устройствами, что означает, что в отличие от ведущих-ведомых систем, где один контроллер направляет поток данных, на шине CAN любой узел может инициировать связь. Кроме того, новые узлы могут быть добавлены в сеть в любом месте между конечными точками, если новые устройства не имеют идентификатора, который конфликтует с существующими устройствами в сети. В случае QNET Mechatronic Interfacing новое устройство может быть подключено к линиям CAN HI и CAN LO в любой точке соединительного кабеля, и оно может общаться со всеми устройствами, в настоящее время подключенными к сети.

Кадры CAN

Данные передаются по шине CAN в виде дискретных сообщений или кадров. Кадры состоят из нескольких разделов: Идентификация, Управление, Данные, Циклический проверочный код (CRC) и Конец кадра.

Идентификация. Существует два различных формата, которые может принимать фрейм данных CAN. Первый – это базовый формат, в котором идентификация передающего устройства хранится только в битах 2-12 фрейма. Этот 11-битный формат идентификатора допускает 2048 уникальных значений идентификатора. Некоторые системы CAN требуют дополнительной гибкости при идентификации характера фреймов. Это обеспечивается фреймом расширенного формата. В этом формате фрейма бит расширения идентификатора (IDE) устанавливается в 1, указывая, что фрейм находится в расширенном формате. Затем за IDE следуют дополнительные 18 бит битов идентификации. В фрейме расширенного формата сообщения могут быть идентифицированы по их 11-битному стандартному идентификатору (SID), их 18-битному расширенному идентификатору (EID) или обоим.

Управление. Основной частью информации в разделе управления фрейма является длина данных. Это 4-битное значение, которое указывает, сколько байтов данных будет следовать. Это важно, поскольку

ку данные не разделены и отправляются в простом последовательном двоичном формате. Это значение будет контролировать, сколько из следующих байтов обрабатываются как данные.

Данные. Каждый кадр CAN может содержать от 0 до 8 байт данных. Именно здесь фактическая информация передается между устройствами на шине CAN. Стоит отметить, что даже несмотря на то, что 4-битное значение управления длиной данных может позволить сообщению указывать до 15 байт данных, стандарт CAN не допускает отправки более 8 байт данных в одном кадре ни при каких обстоятельствах.

Циклический проверочный код представляет собой 15-битный блок данных, который вычисляется из битов данных с использованием полинома и который может использоваться для обнаружения ошибок при передаче данных. Поскольку полином, используемый для генерации CRC во время передачи, также известен получателю, получатель может вычислить делитель, используемый для генерации CRC и проверить, что данные и данные CRC совпадают без ошибок.

Конец кадра. Кадр закрывается семью битами, все из которых должны иметь значение 1. Это указывает другим узлам CAN, что кадр завершен и шина вскоре будет простаивать.

Арбитраж

Отсутствие единого ведущего устройства, определяющего, какое устройство может передавать в данный момент времени, создает ряд проблем. Необходимо избегать случаев, когда несколько устройств одновременно пытаются передавать данные. Решением этой проблемы в стандарте CAN является процесс, называемый арбитражем. Когда линии шины становятся свободными после окончания сообщения, все устройства, ожидающие передачи, отправляют стартовый бит, за которым следует их идентификатор. По мере записи каждого бита идентификации устройство проверяет, соответствует ли состояние шины значению, которое они в данный момент записывают. Если в какой-либо момент устройство пытается записать рецессивный бит (логическая 1), но видит, что шина остается в доминирующем состоянии (логический 0), оно знает, что другое устройство с более низким значением идентификатора также передает данные, и оно прекращает передачу до конца кадра.

Напомним, что устройство, передающее доминирующий бит, всегда победит устройство, передающее рецессивный бит. Таким образом, устройство с наименьшим значением идентификатора (то есть устройство, которое передает рецессивный бит последним) сре-

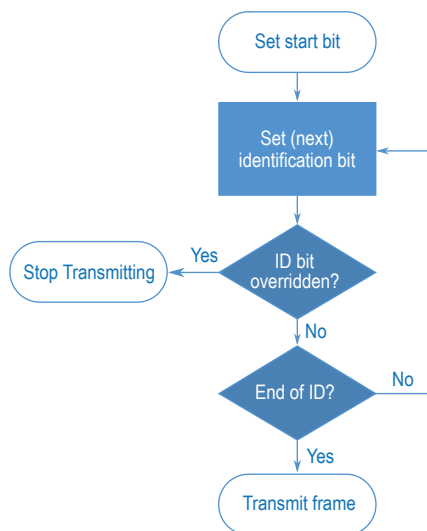


Рис. 1.7. Процесс арбитража CAN

ди устройств, ожидающих передачи, получает арбитраж шины и может передать свой кадр. Процесс арбитража обобщен на рис. 1.7.

Поскольку устройство CAN должно ждать арбитража для передачи сообщения, передача часто происходит с задержкой. Многие устройства CAN имеют несколько буферов передачи с различным приоритетом. Данные загружаются в буфер, и этот буфер помечается как готовый к передаче. Затем устройство передаст сообщение с наивысшим приоритетом в следующий раз, когда оно успешно получит арбитраж.

Фильтрация сообщений

По мере передачи данных по шине CAN они считываются другими устройствами на шине. Приемные устройства сравнивают информацию об идентификаторе в кадре с фильтрами, чтобы решить, предназначено ли сообщение для них. Обычно устройство CAN отклоняет прием сообщения, если какой-либо из битов в идентификаторе сообщения не совпадает с соответствующим битом в фильтре приема. Чипы контроллера CAN на QNET Mechatronic Interfacing имеют два отдельных буфера приема, каждый из которых может фильтровать сообщения по SID, EID или обоим. Буферы приема могут иметь несколько фильтров или могут использовать маски идентификаторов. Если фильтр идентификатора имеет значение маски, он отклонит прием сообщения только в том случае, если один из битов в иденти-

фикаторе сообщения не совпадает соответствующему биту в идентификаторе фильтра, и этот бит в маске установлен на 1.

1.6. Жидкокристаллический дисплей

Тренажер QNET Mechatronic Interfacing снабжен жидкокристаллическим дисплеем (ЖКД) с разрешением 240x320 пикселей, подключенный к микроконтроллеру через параллельную шину данных. Микроконтроллер использует параллельную шину для загрузки данных в память изображений модуля ЖК-дисплея. Поскольку шина данных имеет 8 параллельных линий, один полный байт информации может быть передан за каждый такт. Микроконтроллер использует процесс, называемый прямым доступом к памяти, или DMA, для считывания данных, поступающих из LabVIEW™, непосредственно в память ЖК-дисплея без промежуточного сохранения их в системной памяти. Эта лабораторная работа сосредоточена на VI, который считывает данные изображения из файла PNG и разделяет их на простые данные изображения в форме, аналогичной той, в которой они будут храниться во встроенной системной памяти. Затем эти данные можно обрабатывать различными способами, прежде чем они будут переданы на ЖК-дисплей.

ЖК-дисплеи

ЖК-дисплеи – это плоские дисплеи, которые используют свойства жидких кристаллов изменять поляризацию света, который излучается каждым пикселем дисплея для отображения изображения. Как правило, каждый пиксель ЖК-дисплея состоит из слоя кристаллических молекул, зажатых между двумя листами полимера с микроскопическими канавками. Канавки обычно ориентированы под углом 90° относительно друг друга. Перед приложением электрического поля канавки полностью изменяют выравнивание молекул кристалла, что приводит к плавному повороту на 90° в слое жидкого кристалла. Если поляризационный фильтр добавляется под слоем полимера и кристалла, индуцированный поворот в кристалле способен вращать плоскость поляризации света, который передается через слой. Если дополнительный поляризационный фильтр добавляется над слоем кристалла под углом 90° к фильтру ниже, то поворот в слое кристалла позволяет свету беспрепятственно проходить через обе поляризованные поверхности. При приложении электрического поля к материалу молекулы начинают выравниваться с полем, и их скручивающее воздействие на свет, проходящий через материал, уменьшается. Поэтому, если приложено слабое поле, материал выглядит в основном светлым, тогда как сильное приложенное поле приводит к тому, что материал выглядит темным.

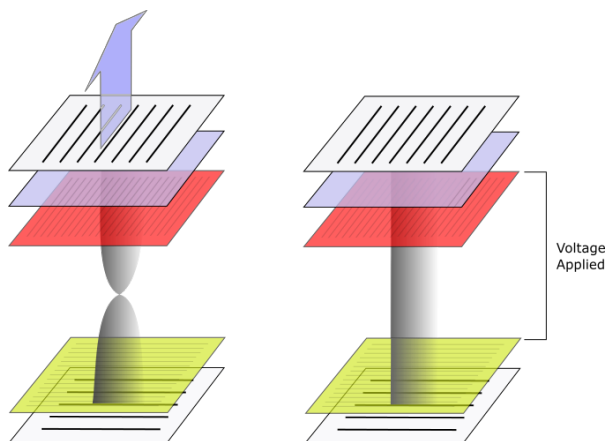


Рис. 1.8. ЖК-ячейка вращает плоскость поляризации света, чтобы позволить ему пройти через фильтр

Каждый пиксель цветного (RGB) ЖК-дисплея, который используется в QNET Mechatronic Interfacing, имеет три отдельных «сэндвича» с цветным фильтром, добавленным в верхнюю часть каждого компонента. Это позволяет контроллеру задавать определенный цвет проходящего через каждый пиксель света, управляя силой электрического поля, воздействующего на каждый из трёх слоев кристаллов.

Данные изображения

Информация, из которой состоит изображение, хранится в памяти в виде серии значений пикселей. Наиболее распространенный способ кодирования цветовой информации в файле называется ARGB или Alpha, Red, Green, Blue. В этом формате значение для каждого пикселя хранится как комбинация альфа-значения или прозрачности для цвета, за которым следуют значения интенсивности красного, зеленого и синего компонентов цвета. В большинстве приложений данные изображения будут храниться в памяти как ряд байтов данных, где каждый блок из четырех байтов представляет цветное значение одного пикселя, как показано на рис. 1.9.

В этом формате каждый пиксель представлен 32 битами информации. Таким образом, говорят, что изображение имеет 32-битную глубину цвета. Данные для изображения хранятся в ряде смежных байтов, как показано на рис. 1.10.

При отображении пиксели со значением альфа ниже 255 будут иметь сохраненные цветовые значения, смешанные с фоновым цве-

Pixel Data																																
Byte number	3								2								1								0							
	Alpha								Red								Green								Blue							
Channel																																
Bit number	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

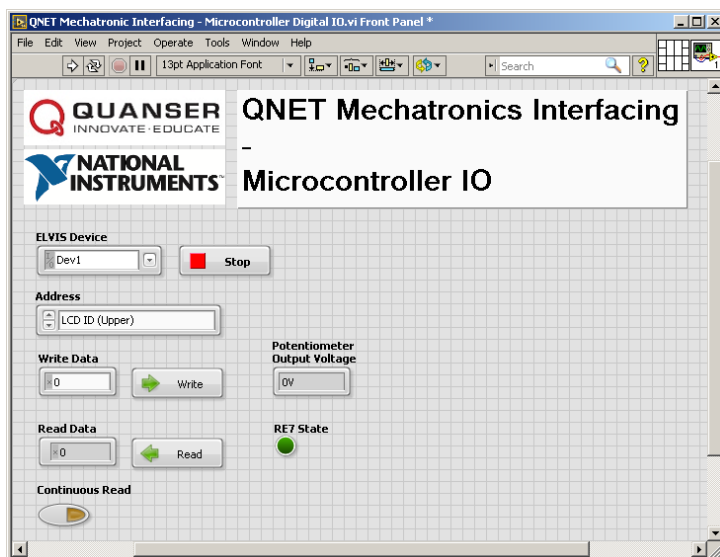
Рис. 1.9. Формат данных пикселей

		Image Data																															
Pixel number		0				1				2				3				4				...				n - 1				n - 2			
																																	
Channel		A	R	G	B	A	R	G	B	A	R	G	B	A	R	G	B	A	R	G	B	A	R	G	B	A	R	G	B	A	R	G	B

Рис. 1.10. Формат данных изображения

том. Если прозрачность не требуется, как во многих ЖК-дисплеях, данные изображения, отправляемые на дисплей, могут полностью отказаться от альфа-компонента. В этом случае остаются только 24 бита, относящиеся к RGB. Это называется 24-битным цветом.

Кроме того, многие дисплеи в действительности не имеют 256 различных градиентов цвета для каждого цвета в пикселе. Конкретный дисплей может взять 8 бит данных, относящихся к каждому цвету, и отбросить один или несколько наименее значимых битов. Например, ЖК-дисплей тренажера использует только 6 бит данных для каждого цвета. В этом случае остается только 18 бит значимых цветовых данных, что означает, что этот ЖК-дисплей имеет 18-битную глубину цвета.



*Рис. 2.2. Виртуальный прибор
для работы с портами микроконтроллера*

2. Убедитесь, что выбрано правильное устройство ELVIS, и запустите VI.

3. Поверните потенциометр против часовой стрелки до упора.

4. В поле Address выберите PORTE. Нажмите кнопку **Read**, чтобы прочитать текущее значение регистра PORTE.

5. В поле Address выберите LATE. Нажмите кнопку **Read**, чтобы прочитать текущее значение регистра LATE. Соответствует ли текущее состояние контактов в порту E значениям в регистре защелки? Учитывая, что значение регистра с тремя состояниями TRISE установлено на 0xFF70, имеет ли это смысл? Почему да или почему нет?

6. Снова выберите PORTE, введите 0x000C в Write Data и нажмите кнопку **Write**. Считайте обновленные значения PORTE и LATE. Какие значения изменились?

7. Порядок, который следует соблюдать при обновлении выходных значений, заключается в считывании текущего зафиксированного значения, установке или очистке битов по мере необходимости и записи результата в регистр порта. Учитывая последнее значение, считанное с LATE, что следует записать в PORTE, чтобы установить RE7 в состояние HIGH, не изменяя состояние других контактов?

8. Запишите найденное ранее значение в PORTE. Индикатор состояния RE7 теперь должен быть включен. Это указывает на то, что микроконтроллер установил линию RE7 в состояние HIGH.

9. Запишите 0x0000 в PORTE, чтобы очистить все ранее установленные значения. Убедитесь, что индикатор состояния линии RE7 не активен.

Цифровой вход

1. Считайте текущее значение ANSELE. Какие из входов, подключенных к потенциометру (напрямую или через триггер Шмитта), настроены как цифровые входы?

2. Считайте и запишите значение PORTE, сначала повернув потенциометр по часовой стрелке до упора, затем повернув потенциометр до упора против часовой стрелки. Какие биты изменили значение? Соответствует ли это входным данным, определенным для настройки в качестве цифровых входов?

3. Снова поверните потенциометр до упора по часовой стрелке, чтобы выходное напряжение потенциометра считывалось как 0 В. Нажмите кнопку **Continuous Read** (непрерывное чтение), чтобы данные считывания для PORTE постоянно обновлялись. Используя потенциометр, медленно изменяйте выходное напряжение потенциометра от 0 В до 3.3 В. При каком напряжении RE5 и RE8 перешли из состояния LOW в HIGH? Может потребоваться повторить процесс, чтобы проверить согласованное напряжение перехода.

4. Повторите процесс регулировки выходного напряжения потенциометра, на этот раз перейдя от 3.3 В до 0 В. При каких напряжениях RE5 и RE8 переходят из состояния HIGH в LOW?

5. На основе полученных значений переходных напряжений количественно оцените гистерезис в RE5 и RE8.

6. Учитывая гистерезис как с внешним триггером Шмитта, так и без него, какова будет способность подавления шума?

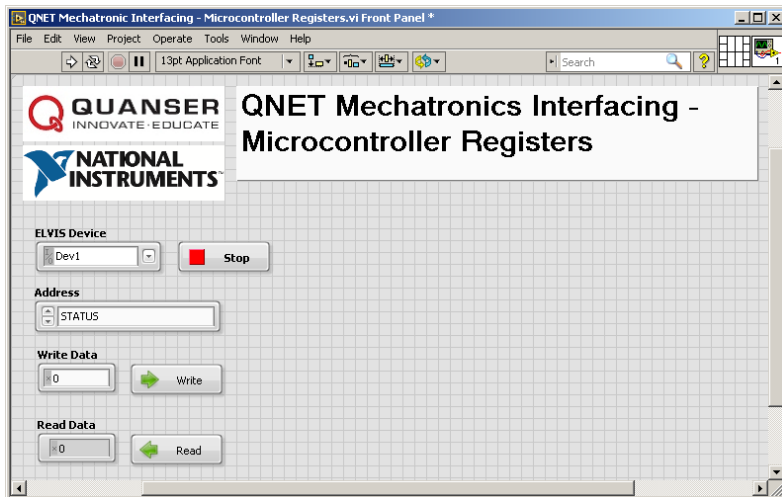
2.2. Основы работы с интерфейсом SPI

Перед выполнением лабораторной работы необходимо ознакомиться с руководством Microchip по работе с SPI[11].

Найдите раздел, описывающий регистр состояния и управления SPI SPIxSTAT. Над каждым битом в регистре находится информация об этом бите, включая значение по умолчанию при включении питания. Какие биты необходимо изменить, чтобы включить связь через SPI? Если начальное значение не изменилось после запуска, какое значение необходимо записать в регистр SPI1STAT?

Ручное управление SPI

1. Откройте «QNET Mechatronic Interfacing - Microcontroller Registers.vi» (рис. 2.3).
2. Убедитесь, что выбрано правильное устройство ELVIS и запустите VI.
3. В поле Address выберите SPI1STAT. Введите найденное ранее значение включения в поле Write Data и нажмите **Write**. Теперь модуль SPI должен быть включен. Вы можете нажать **Read**, чтобы убедиться, что значение было записано правильно.
4. Линия выбора микросхемы для трехцветного светодиода – RK1. Прошивка микроконтроллера настроена так, что при запуске значение LATK устанавливается на 0xE002. Какое значение необходимо записать в PORTK, чтобы перевести нужную линию выбора микросхемы в низкий уровень? Выберите PORTK в поле Address, введите значение в поле Write данных и нажмите **Write**.
5. Теперь, когда модуль SPI активен и выбран трехцветный светодиод, данные можно отправить на устройство, записав 8-битное значение в SPI1BUF. После завершения записи регистра модуль SPI переместит это значение в буфер TX и выведет данные, синхронизируя SCLK. Выберите SPI1BUF в поле Address и запишите 0x7F в буфер SPI.
6. Устройства SPI обычно не обрабатывают полученные команды, пока линия CS не вернется в состояние ожидания (высокий уровень).



*Рис. 2.3. Виртуальный прибор
для работы с управляющими регистрами МК*

Для этого запишите 0xE002 в PORTK. Какое влияние это оказало на световой выход трехцветного светодиода?

7. Повторите шаги выше, очистив линию CS, записав значение и сбросив линию CS. Однако на этот раз запишите 0x00 в SPI1BUF. Какой эффект это оказало на светодиод?

8. Каждый из трех цветов светодиода управляется отдельным цифровым SPI-потенциометром (программно управляемым переменным резистором). Повторите шаги выше, чтобы снова записать 0x00 в светодиод. Исходя из результатов, какова вероятная конфигурация трех устройств SPI, управляющих цветами светодиода? Какое преимущество имеет эта конфигурация?

9. Повторите процесс записи в линию SPI. На этот раз после выбора линии CS светодиода запишите следующие значения в SPI1BUF без сброса линии CS между ними: 0x00, 0x7F, 0x7F. Сбросьте линию CS. Светодиод теперь должен быть сине-зеленым. Учитывая эту информацию, как можно сделать светодиод фиолетовым?

10. Попробуйте разные комбинации значений SPI для получения различных цветов.

11. Запишите 0x0000 в SPI1STAT, чтобы отключить модуль SPI.

12. Нажмите кнопку **Stop**, чтобы остановить VI.

Автоматизированное управление SPI

Учитывая, что ручная настройка регистров по отдельности является повторяющейся и неэффективной, следующим шагом будет создание виртуального инструмента для автоматизации процесса.

Модуль «QNET Mechatronic Interfacing – SPI» (рис. 2.4) имеет три числовых ползунка, которые будут использоваться для установки значений, записываемых на каждый потенциометр по SPI.

1. Откройте «QNET Mechatronic Interfacing – SPI.vi»

2. Откройте блок-схему LabVIEW™ для VI. Общая схема функции уже реализована, включая блоки последовательности ввода-вывода данных, которые устанавливают SPI1STAT для включения SPI при запуске и отключения его после нажатия кнопки Stop. Необходимо добавить только код для управления связью SPI. На рис. 2.5 показано начальное состояние VI с некоторыми важными помеченными компонентами.

3. Начните создавать функцию управления SPI, скопировав один из блоков последовательности ввода-вывода данных вместе со всеми его константами внутрь цикла while и подключив его к туннелю данных на левой стороне цикла while. Обновите константу адреса на PORTK и установите константу записи данных на 0xE000, как показано на рис. 2.6. Этот блок устанавливает линию CS на низкий уровень, чтобы выбрать светодиод, так же, как это делалось вручную ранее.

4. Скопируйте блок последовательности ввода-вывода данных еще четыре раза и подключите их после блока, который был только что добавлен.

5. Измените адрес для 2-го, 3-го и 4-го блока последовательности ввода-вывода данных на SPI1BUF и подключите вход записи данных на блоке к элементам управления «Red», «Green» и «Blue» в нижней части цикла.

6. Установите адрес для 5-го блока последовательности ввода-вывода данных на PORTK, аналогично 1-му блоку, но установите данные записи для этого блока на 0xE002. Это сбросит линию CS в конце каждой записи и даст команду потенциометрам SPI обновиться. Подключите этот последний блок к блоку проверки ошибок на правой стороне цикла, чтобы завершить поток управляющего кластера через VI. На рис. 2.7 показан цикл while, как он должен выглядеть, когда все соединения выполнены правильно.

7. Вернитесь на переднюю панель VI. Убедитесь, что выбрано правильное устройство ELVIS, а диаграмма прибора не содержит ошибок. Запустите VI.

8. Отрегулируйте значения на трех ползунках и убедитесь, что они управляют светодиодом правильного цвета.

9. Понаблюдайте и опишите, что происходит, когда все ползунки установлены до 255. При каком значении каждый цвет становится ярче? Какие значения необходимо установить для каждого цвета, чтобы создать белый свет средней интенсивности?

10. Установите все три цветовых ползунка на ноль и нажмите кнопку **Stop**, чтобы остановить VI.

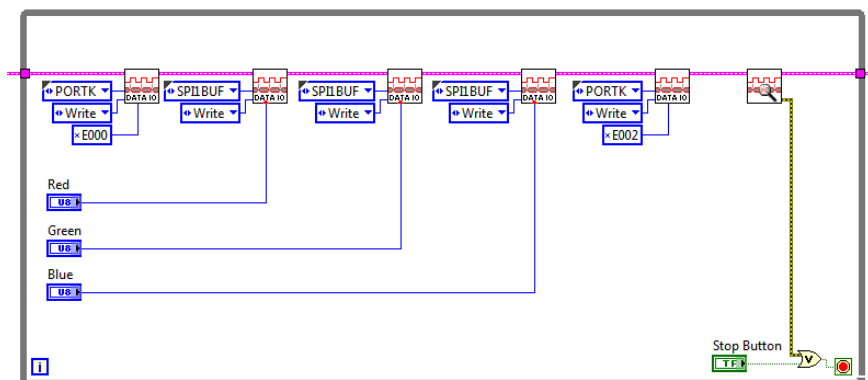


Рис. 2.7. Завершенный цикл while

2.3. Работа с устройствами памяти

Найдите в справочном руководстве по SPI [11] рис. 1.1, который описывает поведение модуля в стандартном режиме. Поток данных через модуль SPI можно проследить на этой схеме. Запись данных в SPI1BUF заставляет модуль SPI смещать эти данные в буфер передачи SPI1TXB. Запись данных в этот буфер сообщает модулю, что есть данные, готовые к отправке. Затем данные перемещаются в сдвиговый регистр SPI1SR. На каждом такте старший значащий бит (MSB) данных в сдвиговом регистре записывается в SDO, а входное состояние SDI сдвигается как новый младший значащий бит.

Ручное чтение SPI

1. Откройте «QNET Mechatronic Interfacing - Microcontroller Registers.vi» (рис. 2.3).

2. Убедитесь, что выбрано правильное устройство ELVIS. Запустите VI.

3. В разделе Address выберите SPI1STAT. Запишите 0x8000 в этот регистр, чтобы активировать модуль SPI.

4. Чтобы инициировать передачу данных по линиям SPI, данные должны быть записаны на MOSI. Выберите SPI1BUF и запишите 8-битное значение (от 0x00 до 0xFF) в этот регистр, чтобы запустить модуль SPI для начала связи.

5. Прочитайте обновленное значение SPI1STAT. Ссылаясь на руководство по SPI, что это значение указывает на состояние модуля SPI? (В руководстве этот регистр упоминается более обобщенно как SPIxSTAT).

6. Прочитайте SPI1BUF. SPI1BUF – это память, сопоставленная с SPI1TXB при записи и SPI1RXB при чтении. Таким образом, значение, считанное из SPI1BUF, должно быть полученными данными, а не данными, записанными в этот буфер ранее.

Поскольку ни одна линия выбора микросхемы устройства SPI не была активна, фактические данные в модуль не поступают. Вместо этого вход будет считывать все 1 и возвращать 0xFF.

7. Действие чтения или записи данных в регистрах иногда приводит к изменениям в других местах, поскольку модуль записывает информацию о состоянии. Снова прочитайте SPI1STAT. Что произошло со значением бита индикатора SPIRBF? Объясните, почему оно изменилось или не изменилось. Обратите внимание, что модуль SPI работает в стандартном буферном режиме.

8. Изучите по техническому описанию микросхемы 23K640[12] команды для работы с этим устройством через SPI (таблица 2-1).

9. Ознакомьтесь с разделом 2.5 технического описания, описывающим процедуру чтения регистра состояния микросхемы ОЗУ.

10. Перед отправкой любых команд в микросхему ОЗУ ее необходимо выбрать. Вернитесь к VI и прочитайте LATA. Регистр должен быть установлен на 0x00C0 или 0x00E0 по умолчанию. Запишите 0x0080 в PORTA, чтобы очистить бит 7 LATA и установить линию выбора микросхемы RAM на низкий уровень. Это сообщает микросхеме RAM, что данные, передаваемые на MOSI, должны быть прочитаны и интерпретированы.

11. Запишите 0x05 в SPI1BUF. Какую операцию выполняет данная команда?

12. Перед получением данных необходимо прочитать буфер приема, чтобы очистить SPIRBF. Если этого не сделать, данные, полученные после заполнения буфера, будут отброшены, и модуль SPI установит SPIROV, флаг переполнения приема SPI в SPI1STAT. Если это произойдет, модуль SPI не будет получать никаких новых данных, пока буфер не будет прочитан, и SPIROV не будет очищен путем записи 0x8000 в SPI1STAT.

13. Прочтите SPI1BUF. Он должен вернуть 0xFF.

14. На рис. 2.7 в техническом описании ОЗУ показана временная последовательность для чтения регистра состояния. Верхняя строка показывает, что линия выбора остается низкой все время. После получения инструкции на SI чипу требуется еще 8 тактов для отправки данных обратно, в течение которых чипу неважно, какое значение получено на SI.

15. Поскольку неактивное состояние линий данных HIGH, запишите 0xFF в SPI1BUF, чтобы запустить требуемые такты.

16. Чип ОЗУ должен был отреагировать на такты, записав данные на MISO. Снова прочитайте SPI1BUF. Какое значение было возвращено для статуса чипа ОЗУ? Что означает этот статус согласно таблице 2-2 в техническом описании 23K640?

17. Обратите внимание, что биты 1–5 регистра состояния не изменяются и имеют как нулевые, так и единичные биты. Почему это может быть полезно при проверке успешной связи?

18. Запишите 0x00C0 в PORTA, чтобы отключить выбор чипа RAM. Это сообщает чипу, что команда чтения регистра состояния, которую он получил, завершена, и оставляет линии SPI открытыми для последующей связи.

19. Нажмите кнопку **Stop**, чтобы остановить VI.

Чтение/запись ОЗУ

1. Откройте QNET Mechatronic Interfacing – Memory.vi (рис. 2.8).

Этот VI автоматизирует процесс активации правильной линии выбора микросхемы в соответствии с типом выбранного запоминающего устройства в поле Memory Device. Затем он последовательно загружает команды из списка команд из SPI CMD List в SPI1BUF и считывает ответ после каждой команды. Обмен данными инициируется при нажатии кнопки **Write**. После завершения обмена VI освобождаст линию выбора микросхемы.

2. Убедитесь, что выбрано правильное устройство ELVIS. Запустите VI.

3. Убедитесь, что в поле Memory Device выбрано ОЗУ(RAM).

4. В первом слоте списка команд SPI введите значение 0x05. Ниже введите значение 0xFF. Все остальные слоты в списке команд должны оставаться серыми, как показано на рис. 2.9. Это должно быть

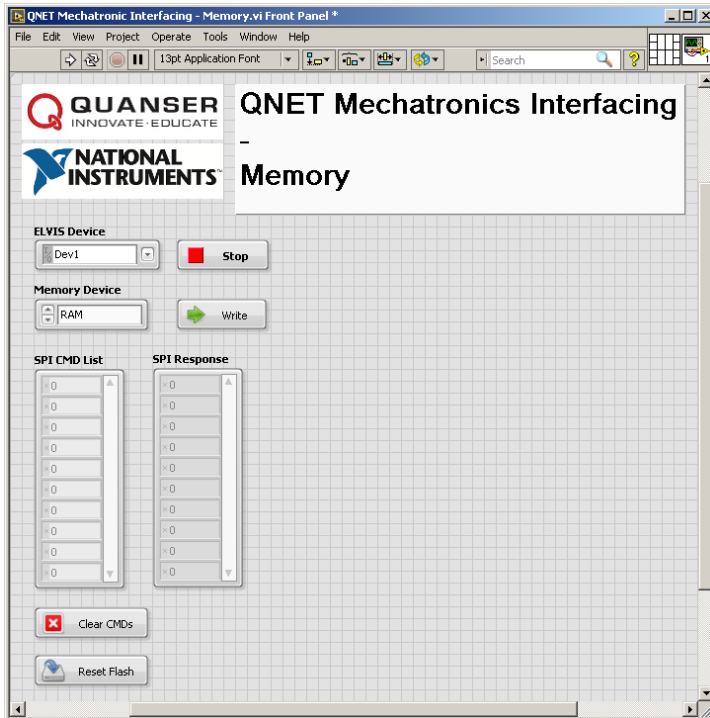


Рис. 2.8. Виртуальный прибор
для работы с запоминающими устройствами

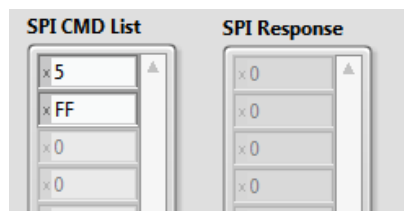


Рис. 2.9. Список команд для чтения регистра состояния ОЗУ

вам знакомо, поскольку это та же процедура для чтения битов состояния ОЗУ, которая была выполнена вручную ранее.

5. Нажмите **Write** и просмотрите значения, возвращаемые в SPI Response. Каждое значение в SPI Response соответствует ответу, полученному на команду рядом с ним. Это также должно быть знакомо, поскольку микросхема ОЗУ должна возвращать те же значения, что и при ручном чтении.

6. Теперь, когда связь проверена, фактические данные могут быть считаны из микросхемы ОЗУ. Установите значения в SPI CMD List на 0x03, 0x00, 0x00, 0xFF, 0xFF, 0xFF, как показано на рис. 2.10, и нажмите **Write**.

7. Какие значения в ответе соответствуют данным, считанным из микросхемы ОЗУ (используйте рис. 2-5 в техническом описании 23K640 для ответа)? С какого адреса были считаны эти данные?

8. Данные, возвращаемые чипом RAM, будут случайными числами, поскольку фактические данные не были записаны в чип с момента его включения. Запишите данные в чип, написав следующие команды в списке команд SPI: 0x02, 0x00, 0x01, 0x11, 0x22, 0x33 и нажав **Write**.

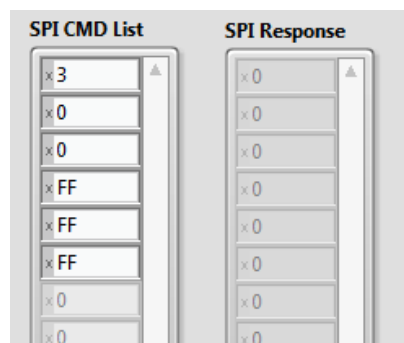


Рис. 2.10. Список команд для чтения данных ОЗУ

9. Какие значения необходимо записать в список команд SPI, чтобы прочитать 5 байт данных, начиная с адреса 0x0000? Отправьте соответствующие команды, введя их в списке команд SPI

10. В разделе SPI response должно быть три байта пустых ответов, за которыми следуют 5 байтов данных. Первый и последний байты будут случайными, а средние три байта будут 0x11, 0x22 и 0x33, как было написано ранее.

11. Нажмите **Clear CMDs**, чтобы очистить массив команд. Чипы RAM допускают произвольную перезапись данных. Повторите шаги для записи значений в чип ОЗУ, на этот раз используя команды 0x02, 0x00, 0x02, 0xAA, 0xBB, 0xCC.

12. Повторите шаги для чтения первых 5 байтов, начиная с 0x0000. Какие значения данных были возвращены? Были ли данные успешно перезаписаны?

13. Нажмите **Clear CMDs**, чтобы очистить список команд.

Чтение/запись Flash

1. Перед началом работы с флэш-памятью ознакомьтесь с техническим описанием микросхемы S25FL164K[13].

2. Нажмите **Reset Flash**, чтобы подготовить флэш-чип.

3. В поле Memory Device выберите Flash.

4. Управление флэш-чипом значительно сложнее, чем чипом RAM, поскольку S25FL164K имеет более 30 уникальных команд в отличие от 4 команд на чипе RAM. Однако после определения правильной команды процесс связи через SPI идентичен. Как и в случае с чипом RAM, первым шагом является проверка успешной связи путем считывания известного значения, такого как регистр состояния или идентификатора. Введите командную серию 0x9F, 0xFF, 0xFF, 0xFF, чтобы прочитать информацию об идентификаторе устройства. Какие значения были возвращены? Определите, ссылаясь на раздел 7.6.1 в техническом описании, указывает ли возвращаемое значение на правильную модель устройства?

5. Из раздела 9.3.1 технического описания следует, что флэш-чип использует ту же команду, что и чип ОЗУ, 0x03, для чтения данных. Однако есть одно отличие в структуре последовательности команд SPI. Чем отличается чтение из флэш-чипа? Какая команда будет читать первые три байта флэш-данных, начиная с адреса 0x000000?

6. Введите команды для чтения первых трех байтов данных в списке команд SPI и нажмите **Write**. Чем отличаются данные, изначально сохраненные в пустой флэш-памяти, от данных, сохраненных в ОЗУ?

7. В техническом описании S25FL164K найдите раздел, который относится к команде Write Enable. Каково значение команды и когда ее нужно отправлять на флэш-чип?

8. Отправьте команду Write Enable.

9. Теперь, когда запись на чип разрешена, введите последовательность команд 0x02, 0x00, 0x00, 0x01, 0x11, 0x22, 0x33 в списке SPI CMD и нажмите Write. Эта последовательность команд запишет данные в ячейки данных 0x000001-0x000003.

10. Повторите команду чтения, использованную ранее, однако на этот раз считайте первые 3 байта данных, начиная с 0x000001. Убедитесь, что возвращенные данные соответствуют данным, записанным на предыдущем шаге.

11. Снова отправьте команду Write Enable. После того, как запись на чип разрешена, введите последовательность команд 0x02, 0x00, 0x00, 0x02, 0xAA, 0xBB, 0xCC в списке SPI CMD и нажмите Write. На основе двух команд записи, выполненных до сих пор, и справочной информации, какие данные теперь должны содержаться в первых 5 байтах флэш-памяти (начиная с 0x000000)?

12. Введите серию команд 0x03, 0x00, 0x00, 0x00, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF в списке команд SPI и нажмите Write. Учитывая, что по сути одни и те же команды чтения и записи были выполнены как в ОЗУ, так и во флэш-памяти, чем текущее состояние флэш-памяти отличается от последнего значения, считанного из ОЗУ? Почему?

13. Нажмите кнопку **Stop**, чтобы остановить VI. Не закрывайте VI, так как он еще будет использоваться.

Изменчивость данных

1. Выключите питание QNET Mechatronic Interfacing с помощью выключателя питания справа от платы QNET. Подождите немного, прежде чем снова включить питание.

2. Запустите VI и считайте первые 5 байтов (начиная с адреса 0x0000/0x000000) данных в ОЗУ и флэш-памяти. Изменились ли они с момента выключения и включения питания? Почему ожидалось такое поведение?

Стирание флэш-данных

1. В техническом описании S25FL164K найдите раздел, описывающий команду Sector Erase. Какой наименьший блок данных можно стереть на этом чипе? Какие команды нужно отправить на чип, чтобы стереть данные, хранящиеся в первых 5 байтах флэш-памяти?

Таблица 2.1

Применение	Тип памяти	Причина выбора
Память программы микроконтроллера		
Данные регистра микроконтроллера		
Буфер данных GPS		
Данные конфигурации GPS		
Домашние координаты мобильного робота		
Данные о препятствиях мобильного робота		
Данные захвата изображения камерой		
Хранение изображений камерой		
Съемное хранилище (флеш-накопители)		

2. Выполните стирание сектора по адресу сектора 0x000000. После этого считайте данные в первых 5 байтах флэш-памяти и убедитесь, что данные действительно были стерты.

Применение запоминающих устройств

В столбце 1 табл. 2.1 перечислены несколько потенциальных вариантов применения устройств памяти. Заполните таблицу, указав в столбце 2, что лучше подойдет для каждого случая – ОЗУ или флэш-память, и указав в столбце 3 причину выбора этой технологии памяти для этого применения.

2.4. Работа с интерфейсом I2C

Эта лабораторная работа посвящена взаимодействию с датчиком температуры MCP9801. Однако процесс, описанный для взаимодействия с датчиком через интерфейс I2C, применим ко всем датчикам с аналогичным интерфейсом, имеющимся в QNET Mechatronic Interfacing.

Ручное управление I2C

1. Откройте «QNET Mechatronic Interfacing - Microcontroller Registers.vi» (рис.2.3).

2. Убедитесь, что выбрано правильное устройство ELVIS. Запустите VI.

3. QNET Mechatronic Interfacing запускается с включенным модулем I2C I2C2 и готовым к началу взаимодействия. В поле Address выберите I2C2CON и нажмите **Read**. Убедитесь, что он возвращает 0xB000.

4. В поле Write Data введите 0xB001 и нажмите **Write**. Опираясь на информацию из руководства по интерфейсу dsPIC I2C[14] для регистра управления I2CxCON определите, какой эффект это изменение окажет на шину I2C?

5. Выберите регистр состояния I2C I2C2STAT и считайте его текущее значение. Возвращенное значение должно быть 0x0008. Если это не так, нажмите кнопку сброса системы и повторите предыдущие шаги. Ссылаясь на раздел справки I2C по I2CxSTAT определите, какой бит указывает, успешно ли запущена связь I2C? Как этот бит указывает на текущий статус модуля I2C?

6. В техническом описании MCP9801[15] найдите информацию о байте адреса. Каков общий 7-битный адрес для этого устройства, учитывая, что A0, A1 и A2 установлены в 0? Какими будут 8-битные адреса для чтения и записи на это устройство?

7. Для простоты регистр буфера передачи I2C2TRN и регистр буфера приема I2C2RCV были объединены в один регистр, называемый I2C2TXRX. Запись в этот регистр записывает в буфер передачи, чтение из этого регистра возвращает значение, содержащееся в буфере приема. Выберите I2C2TXRX в поле Address. В поле Write Data введите шестнадцатеричное значение адреса чтения для MCP9801 и нажмите **Write**.

8. Считайте текущее значение I2C2STAT. Возвращаемое значение по-прежнему должно быть 0x0008. Ссылаясь на техническое описание, найдите значение бита состояния ACKSTAT. Что означает, что этот бит не изменился? (Обратите внимание, если I2C2STAT возвращает 0x8008, проверьте значение бита адреса и повторите предыдущий шаг.)

9. Теперь, когда ведомое устройство успешно выбрано по адресу и подтвердило команду чтения, модуль I2C должен выполнить цикл SCL 8 раз и считать данные из SDA. Запишите 0xB008 в I2C2CON. Это устанавливает бит разрешения приема RCEN и сообщает модулю I2C выполнить цикл SCL 8 раз и принять данные в приемный буфер.

10. Считайте значение в I2C2STAT. Возвращаемое значение теперь должно быть 0x000A. Бит 1 (RBF: Receive Buffer Full Status bit) теперь имеет значение 1, указывая на заполнение приемного буфера.

11. Считайте I2C2TXRX. Какое значение было возвращено? По умолчанию первый возвращенный байт будет старшим байтом значения температуры. Используя техническое описание MCP9801 определите, какой температуре соответствует возвращаемое значение?

12. Завершите процесс чтения, записав 0xB030 в I2C2CON. Это устанавливает ACKDT и ACKEN в 1, заставляя модуль I2C отправить бит отрицательного подтверждения (NACK) на прием данных, завершая чтение.

13. Запишите 0xB004 в I2C2CON, чтобы остановить связь I2C.

14. Нажмите кнопку **Stop**, чтобы остановить VI.

Автоматизированное управление I2C

Ручная настройка регистров по отдельности является повторяющейся и неэффективной. Следующий шаг – создание функции для автоматизации процесса. Модуль «QNET Mechatronic Interfacing – I2C.vi» (рис. 2.11) имеет числовой индикатор, отображающий приведенное значение температуры с датчика в реальном времени.

1. Откройте «QNET Mechatronic Interfacing – I2C.vi».

2. Откройте блок-схему прибора. Общая схема функции уже реализована, включая блок Set I2C Pullups, который обрабатывает регулировку сопротивления подтягивания линии данных I2C, и блоки Data IO Sequence, которые записывают в I2C2CON и I2C2TXRX для выбора датчика температуры и записи 0x00 в указатель регистра. Также присутствует формирование выходных данных и кнопка Stop для остановки цикла while. На рис. 2.12 показано начальное состояние VI с некоторыми важными помеченными компонентами.

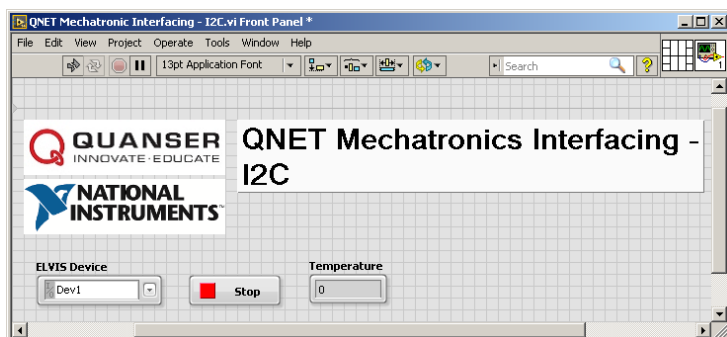


Рис. 2.11. Панель виртуального прибора датчика температуры

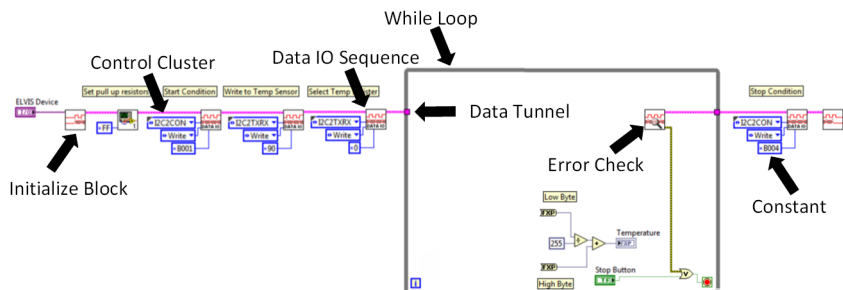


Рис. 2.12. Начальная диаграмма виртуального прибора

4. Начните создавать функцию чтения I2C, скопировав блок Start Condition Data IO Sequence вместе со всеми его константами внутри цикла while и подключив его к туннелю данных на левой стороне цикла while. Обновите константу Write Data на 0xB002, чтобы изменить ее на повторяющееся условие запуска, как показано на рис. 2.13.

5. Далее, как и при ручном чтении данных, адрес должен быть записан в I2C2TXRX. Вставьте еще одну копию блока Data IO Sequence и подключите ее к предыдущему блоку. Измените переменную адреса на I2C2TXRX, и измените константу Write Data на 0x91, чтобы выбрать MCP9801 для чтения данных.

6. Вставьте еще три копии блока Data IO Sequence и подключите их вместе и к предыдущему блоку. Эти три блока будут выполнять фактическое чтение. Измените константы для первого нового блока так, чтобы он записывал 0xB008 в I2C2CON. Напомним, что это запустит процесс чтения. Во втором новом блоке последовательности ввода-вывода данных полностью удалите константу Write Data. Измените адрес на I2C2TXRX и установите константу чтения/записи на Read. Подключите выход Read Data этого блока к входу High Byte в блоки обработки данных температуры. Третий блок последовательности ввода-вывода данных должен быть настроен на запись 0xB010 в I2C2CON, чтобы подтвердить чтение данных.

7. Скопируйте все последние три блока последовательности ввода-вывода данных вместе и вставьте копию между последним блоком последовательности ввода-вывода данных и блоком проверки ошибок и соедините их. Подключите выход Read Data из новой последовательности чтения к входу Low Byte в блоки обработки данных температуры. После того, как все блоки будут размещены и настроены, результирующая диаграмма внутри цикла while должна выглядеть так, как показано на рис. 2.14.

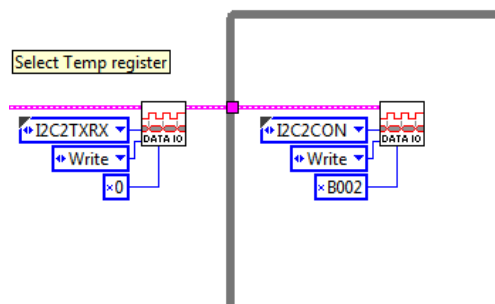


Рис. 2.13. Добавление выбора чипа датчика

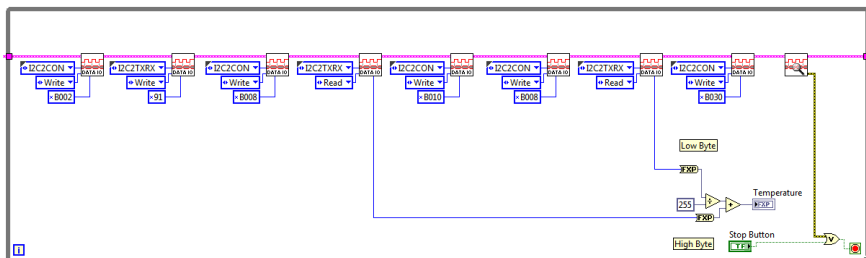


Рис. 2.14. Завершенный цикл while

8. Вернитесь на переднюю панель VI и убедитесь, что выбрано правильное устройство ELVIS. Запустите VI.

9. Понаблюдайте и опишите, что происходит, когда палец удерживается на верхней части датчика температуры I2C. Какой диапазон значений температуры достигается?

10. Объясните, почему данные младшего байта от датчика температуры необходимо разделить на 255 перед добавлением к данным старшего байта. Что представляет собой младший байт, взятый в целом?

11. Нажмите кнопку Stop, чтобы остановить VI.

12. Используя техническое описание MCP9801, напишите в псевдокоде функцию, которая изменяет регистр CONFIG на MCP9801, так что датчик будет работать с 11-битным разрешением, затем считывает регистры температуры и выводит значение температуры. Предположим, что модуль I2C запускается в остановленном состоянии.

2.5. Обмен данными в сети CAN

Тренажер QNET Mechatronic Interfacing имеет два независимых узла CAN, построенных на базе контроллеров MCP2515[16] с интерфейсом SPI и приемопередатчиков MCP2551. В этой лабораторной работе эти два устройства, называемых CAN0 и CAN1, взаимодействуют друг с другом для имитации различных видов фильтрации сообщений, доступных в коммуникации шины CAN.

Обычно шина CAN имеет более 2 устройств, соответственно, все концепции, рассматриваемые в этой лабораторной работе, масштабируются до произвольно большого числа устройств.

Сеть CAN с двумя узлами

1. Откройте «QNET Mechatronic Interfacing – CAN Bus.vi».
2. Убедитесь, что выбрано правильное устройство ELVIS (рис. 2.15).
3. Запустите VI. Подождите, пока устройства CAN будут инициализированы.

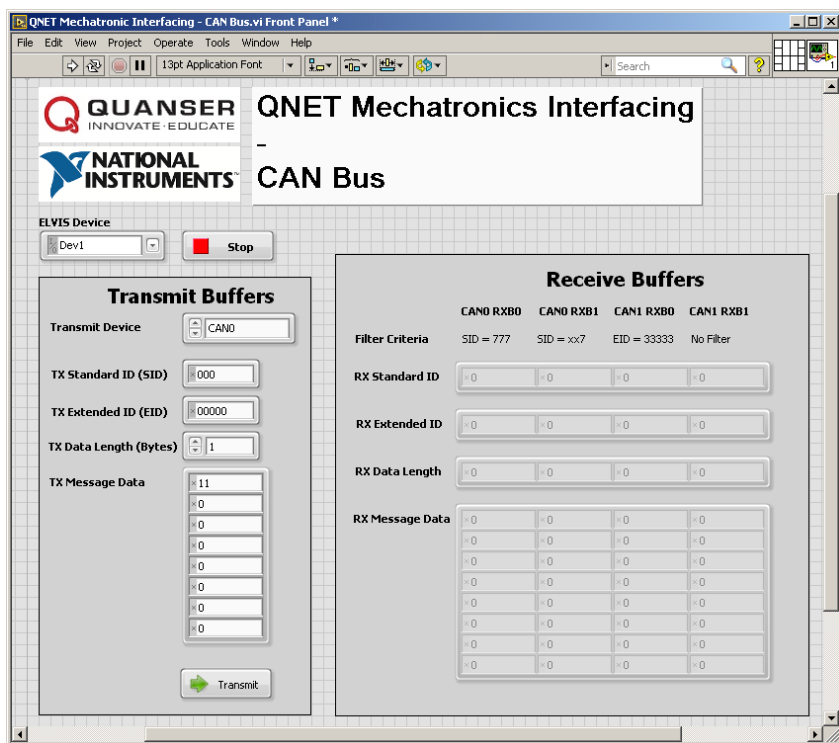


Рис. 2.15. Виртуальный инструмент для работы с сетью CAN

Оба контроллера CAN имеют два приемных буфера, RXB0 и RXB1. После завершения инициализации VI заполнит таблицу приемных буферов текущими данными, хранящимися в каждом из этих буферов. Обратите внимание, что поскольку данные приемного буфера хранятся в энергозависимой памяти, значения, хранящиеся в этих буферах при включении питания, будут случайными.

4. Оставьте элементы управления буфером передачи в их состоянии по умолчанию (передающее устройство CAN0, SID = 0, EID = 0, длина данных – 1 байт, data[0] = 11), нажмите кнопку **Transmit** для передачи. Это приводит к тому, что информация, введенная в разделе Transmit Buffers, загружается в буфер передачи на выбранном устройстве передачи. Затем устанавливается бит запроса на передачу сообщения из этого буфера, инициирующий передачу сообщения устройством, как только ему удастся получить управление шиной.

5. Единственным устройством на шине, которое могло получить сообщение, в этом случае был CAN1. Какой буфер приема получил сообщение? Почему сообщение оказалось в этом буфере?

6. Сколько байтов данных сообщения приема в обновленном буфере приема содержат фактические данные? Что произошло с данными в других байтах буфера данных сообщения?

7. Измените устройство передачи на CAN1 и снова нажмите **Transmit**. Это вызовет отправку того же сообщения, что и раньше, но на этот раз с использованием CAN1 в качестве передатчика. Были ли изменения в значениях в буферах приема? Почему этот результат имеет смысл?

8. Какое значение в буфере передачи должно измениться, чтобы сообщение было получено CAN0 RXB0?

9. Отредактируйте значения в буферах передачи так, чтобы CAN1 передавал 8 бит данных в CAN0 RXB0. Передайте сообщение и убедитесь, что оно было получено правильным буфером.

10. CAN0 RXB1 использует маску идентификатора, так что только 4 младших бита адреса сравниваются с фильтром. Это означает, что не имеет значения, каким будет SID сообщения, если он заканчивается на 0x7. Приведите пример SID, который можно выбрать так, чтобы сообщение было получено CAN0 RXB1.

11. Установите элементы управления буфером передачи таким образом, чтобы CAN1 отправлял сообщение, которое будет получено CAN0 RXB1. Измените длину сообщения и данные сообщения на значения, отличные от значений предыдущего сообщения, чтобы можно было легко определить, что сообщение не было получено неправильным буфером. Нажмите **Transmit** и убедитесь, что сообщение было фактически получено в правильный буфер.

12. Соответствует ли SID сообщений, которые соответствуют требованиям фильтра CAN0 RXB0, требованиям фильтра CAN0 RXB1? Учитывая поведение, наблюдаемое до этого момента, какой вывод можно сделать об относительном приоритете двух буферов приема?

13. До этого момента поле расширенного идентификатора не использовалось. Измените устройство передачи обратно на CAN0. Измените расширенный идентификатор TX на 0x333333 и выберите произвольные значения для длины данных и данных сообщения. Нажмите **Transmit**. Убедитесь, что сообщение было успешно получено CAN1 RXB0.

15. Попробуйте изменить стандартный идентификатор сообщения. Имеет ли значение SID какое-либо влияние на то, получено ли сообщение RXB0 или RXB1?

16. Нажмите кнопку **Stop**, чтобы остановить VI.

Более крупные сети CAN

1. И CAN0, и CAN1 были настроены на проверку либо SID, либо EID, но не обоих одновременно. Учитывая, что SID имеет длину 11 бит, а EID – 18 бит, сколько уникальных идентификаторов сообщений будет доступно для устройства CAN, которое фильтруется как по стандартному, так и по расширенному идентификатору?

2. Если бы внешнее устройство было добавлено к шине CAN, подключенной к CAN0 и CAN1, какой идентификатор устройства привел бы к тому, что сообщения с этого устройства будут приниматься как CAN0 RXB0, так и CAN1 RXB0?

3. Таблица 2.2, взятая из технического описания MCP2515, показывает логику, которая определяет, принят ли идентификатор на основе значений маски и фильтра. Предположим, что к шине подключены два внешних устройства со стандартными идентификаторами 0x333 и 0x2E2. Какие значения фильтра и маски позволят получать сообщения с обоих устройств одним буфером?

Таблица 2.2

Таблица истинности фильтра идентификаторов

Бит маски	Бит фильтра	Бит идентификатора сообщения	Реакция
0	x*	x*	Принять
1	0	0	Принять
1	0	1	Отклонить
1	1	0	Отклонить
1	1	1	Принять

*значение не оказывает влияния

2.6. Формирование изображения на ЖКИ

Эта лабораторная работа посвящена процессу подготовки данных изображения для отображения на ЖК-экране. Виртуальный инструмент отрабатывает процесс передачи данных на ЖК-дисплей автоматически с помощью прямого доступа к памяти. В лабораторной работе будут рассмотрены различные ситуации, при которых данные изображения могут быть неправильно сформированы и/или отформатированы, и способы решения таких проблем.

1. Откройте «QNET Mechatronic Interfacing – LCD.vi».

2. Убедившись, что выбрано правильное устройство ELVIS, запустите VI (рис. 2.16).

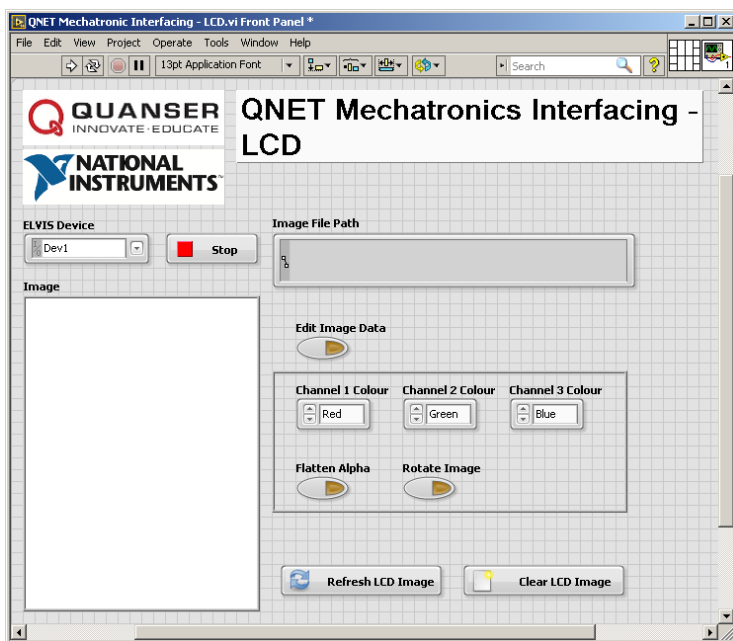


Рис. 2.16. Виртуальный инструмент для работы с ЖКИ

3. В появившемся диалоговом окне выбора файла выберите «LCD Test Image.png» в папке «LCD Images» и нажмите ОК.

4. Не изменяя никаких других настроек в VI, нажмите **Refresh LCD Image**, это запишет неформатированные данные изображения непосредственно на ЖК-экран. Что можно наблюдать в изображении на ЖК-дисплее? Перезаписаны ли какие-либо пиксели на экране более одного раза?

Кнопки **Clear LCD Image** и **Refresh LCD Image** можно использовать для отрисовки пустого экрана и повтора процесса записи изображения.

5. ЖК-дисплей сохраняет данные изображения в формате 24-бит RGB (красный-зеленый-синий). Учитывая, что VI считывает PNG в массив 32-битных пиксельных фрагментов формата ARGB (альфа-красный-зеленый-синий), почему ЖК-дисплей может вести себя так, как вы наблюдали?

6. Нажмите **Edit Image Data**, чтобы включить обработку изображения. По умолчанию все, что делается, это полностью удаляется альфа-байт из каждого пикселя.

Снова нажмите **Refresh LCD Image**. Как удаление альфа-бита повлияло на изображение? Были ли какие-либо пиксели перезаписаны более одного раза? Почему удаление альфа-байта могло привести к тому, что изображение стало выглядеть таким образом?

7. Нажмите **Flatten Alpha**, чтобы настроить значения RGB для каждого пикселя с учетом удаляемого альфа-значения. VI выполняет это путем масштабирования каждого из значений цвета пикселей в соответствии с формулой

$$C_{new} = 255 - \left(\frac{A}{255} \cdot (255 - C_{old}) \right),$$

где C_{old} и C_{new} – исходное и новое значение цвета для одного цвета в данном пикселе, а A – значение альфа для этого пикселя. Это приблизительно соответствует тому, как будет выглядеть изображение с нетронутыми данными альфа при размещении на белом фоне.

8. Нажмите **Refresh LCD Image**, чтобы обновить изображение с добавленными настройками альфа. Это приведет к получению изображения, гораздо более похожего на оригинал, однако изображение все еще не отображается так, как предполагалось.

9. Обратите внимание, что когда изображение рисуется на ЖК-дисплее, оно обновляется снизу вверх. Это происходит потому, что «верх» ЖК-дисплея, то есть пиксели, которые записываются первыми, на самом деле находятся внизу платы. Нажмите **Rotate Image**. Это заставляет VI изменить порядок пиксельной информации. Байт изображения, который изначально был записан первым, теперь последний, и наоборот.

10. Нажмите **Refresh LCD Image**, чтобы обновить изображение с обратным порядком пикселей. Теперь изображение должно отображаться правой стороной вверх. Какой еще эффект оказало изменение порядка байтов на изображение? Почему изменение порядка байтов может вызвать наблюдаемый эффект?

11. Поэкспериментируйте с настройками цвета канала, пока изображение не появится с правильными цветами. В каком порядке должны быть записаны значения цвета на ЖК-дисплей, если данные изображения перевернуты?

12. Продолжайте экспериментировать с настройками цветового канала. Что происходит с изображением, если для любых двух или всех трех каналов выбран один цвет?

13. Нажмите **Stop**, чтобы остановить VI.

14. Используя программу редактирования изображений, например Microsoft Paint, откройте файл «Blank Image.png» в качестве заготовки для создания нового изображения.

Отредактируйте файл таким образом, чтобы получить изображение, которое будет выглядеть правильно при использовании только начального этапа обработки удаления альфа-данных.

16. Запустите VI снова, на этот раз выбрав новое изображение и посмотрите, как изменится внешний вид при деактивации различных процессов редактирования изображения. Какими характеристиками должно обладать изображение, чтобы не быть затронутым проблемами, возникшими ранее?

17*. В некоторых случаях изображение может быть отредактировано после того, как оно считано в память. Это может быть сделано для выделения части изображения или отображения некоторой информации. Учитывая данные изображения сплошного белого цвета (все значения равны 255) в одномерном формате массива BGR, в котором изображение подается на ЖК-дисплей, напишите псевдокод, который изменяет данные изображения, чтобы добавить красный квадрат A пикселей, поперек которого находится x пикселей от левого края экрана и y пикселей от верхней части экрана.

18*. Еще одной распространенной задачей при работе с данными изображений является поиск информации в изображениях. Этот процесс называется обработкой изображений и является ключевым для таких задач, как компьютерное зрение. Для изображения, созданного с помощью упомянутого выше алгоритма (красный квадрат на белом фоне), напишите псевдокод, который определяет местоположение в массиве изображений пикселя в середине квадрата

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. QNET Mechatronic Interfacing for NI ELVIS, User Manual, Quanser Inc., 2016.
2. QNET Mechatronic Interfacing for NI ELVIS, Instructor Workbook, Quanser Inc., 2016.
3. NI Educational Laboratory Virtual Instrumentation Suite II Series (NI ELVIS™ II Series) User Manual, 374629C-01, National Instruments Corporation., June 2011.
4. LabVIEW User Manual April 2003 Edition Part Number 320999E-01.
5. 16-Bit Microcontrollers and Digital Signal Controllers with High-Speed PWM, USB and Advanced Analog, DS70616G, Microchip Technology Inc., Oct 2012.
6. Using the Serial Peripheral Interface to Communicate between Multiple Microcomputers, Application Note AN991/D, Motorola Inc., 1987.
7. UM10204 I2C-bus specification and user manual, Rev. 6, NXP Semiconductors N.V., 4 April 2014.
8. CAN Specification, Version 2.0, Robert Bosch GmbH, Sep. 1991.
9. ISO 11898-1:2024 Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical coding sublayer, Edition 3, 2024-05.
10. ГОСТ Р ИСО 11898-1 – 2015 Транспорт дорожный. МЕСТНАЯ КОНТРОЛЛЕРНАЯ СЕТЬ (CAN), Часть 1: Канальный уровень и передача сигналов, Москва, Стандартинформ, 2016, 42с.
11. Serial Peripheral Interface (SPI), dsPIC33/PIC24 Family Reference Manual, DS70005185A, Microchip Technology Inc., 2014
12. SPI Bus Low-Power Serial SRAM, DS22126E, Microchip Technology Inc., Oct 2011.
13. CMOS 3.0-Volt Flash Non-Volatile Memory Serial Peripheral Interface (SPI) with Multi-I/O, Publication Number S25FL132K_164K_00 Rev.3, Spansion Inc., Oct 16, 2013.
14. Inter-Integrated Circuit™ (I2C™), dsPIC33/PIC24 Family Reference Manual, DS70000195F, Microchip Technology Inc., 2014.
15. 2-Wire High-Accuracy Temperature Sensor, DS21909D, Microchip Technology Inc., 2010.
16. Stand-Alone CAN Controller With SPI™ Interface, DS21801D, Microchip Technology Inc., 2005.

СОДЕРЖАНИЕ

Введение	3
1. Общие сведения об исследуемых интерфейсах	6
1.1. Интерфейс ввода/вывода микроконтроллера	6
1.2. Последовательный периферийный интерфейс	9
1.3. Интерфейс запоминающих устройств	12
1.4. Интерфейс связи интегральных компонентов	14
1.5. Местная сеть контроллеров	19
1.6. Жидкокристаллический дисплей	24
2. Лабораторные работы	27
2.1. Ввод/вывод дискретных сигналов в МК	27
2.2. Основы работы с интерфейсом SPI	29
2.3. Работа с устройствами памяти	34
2.4. Работа с интерфейсом I2C	40
2.5. Обмен данными в сети CAN	44
2.6. Формирование изображения на ЖКИ	47
Библиографический список	51